CS 534 Individual Project Assignment 3 (100 Points)
Due: 11:59 p.m. on 03/29/2026

Project Objective: The goals of this assignment are to help you understand the game concepts and the adversarial search algorithms that we discuss in Week 5, respectively.

Note:

*(1) This project is to be done by **EACH STUDENT** individually. No help besides the textbook, materials, and the instructor/TA should be taken. Copying any answers or part of answers from other sources, including your classmates, will earn you a grade of zero.*
*(2) Your program must be developed and implemented in the PyCharm-like IDE, or 10% of the graded score is deducted. Please check and choose the one from here: https://realpython.com/python-ides-code-editors-guide/, as the suggestion. Note that we **DO NOT** use the Jupyter as the IDE. In this assignment, you can use the codes provided by **games4e.py**.*
(3) Assignments are accepted in their assigned Canvas drop box without penalty if they are received by 11:59PM EST on the due date, or 10% of the graded score is deducted for the late submission per day. Work submitted after one week of its original due date will not be accepted.

Project Deliverables: Submit a **zip** file that includes the **.py files**, i.e., TicTacToeClass.py, TicTacToeGameApp.py, and other **python files/folders (if needed)**, to complete your individual project assignment to Canvas.

In this project, we explore and understand the use of adversarial search algorithms to play the Tic-Tac-Toe game discussed in our class.

(1). Develop and implement a **TicTacToeClass.py** (TTT) class to instantiate a TTT object to play a game between the two players, i.e., Player 'X' and Player 'O'. You can review and investigate the **"Game" class and TicTacToe(Game) in games4e.py**, as the reference and the starting point, provided from **https://github.com/aimacode/aima-python**. Note that you **CAN** develop your own class based upon your understanding.

(2). There are six players that allow human users to choose any two of them to play the game. Those players include:

1. Random Player
2. MiniMax Player
3. Alpha Beta Player
4. Heuristic Alpha Beta Player
5. MCTS Player
6. Query Player

(3). By default, each game is a 3 x 3 board. It means when a player has 3 'X' or 3 'O' in a row, in a column, or in a diagonal, that player is a winner.

(4). For the TTT class, you need to develop and implement the search algorithms that we discussed: **minmax_decision(state, game)**, **alpha_beta_search(state, game)**, **alpha_beta_cutoff_search(state, game)**, and **monte_carlo_tree_search(state, game)**. All of those functions, as the reference and the starting point, can be found in **games4e.py**. Note that you **CAN** develop your own functions for the TTT class based upon your textbook.

(5). Develop and implement a separate python program called **TicTacToeGameApp.py** using the given construct and do the following:

```python
def main():
    pass

if __name__ == "__main__":
    main()
```

(a) Your app will prompt and ask for the human user to enter which search strategy of each player 'X' and 'O' is used, respectively, in the game. See the screenshot below as an example. You can find the corresponding codes for players in **games4e.py**. Please feel free to modify it to meet the requirements.

(b) Your app will then create a TTT object from the **TicTacToeClass.py** based upon the human user's input from (a).

(c) Your game output should show each move of Player 'X' or 'O' in its own turn, respectively, in sequence (see the below example as the reference). The move output can be on a console that would be good enough.

Note that each game has three rounds.

<u>Case 1:</u> If Player 'O' or Player 'X' can win the **FIRST TWO** rounds of a game in a row, the game should terminate immediately without moving on to the 3rd round, displaying the following output:

Player O can win two out of three rounds in the game.
Player O is the winner.

<u>Case 2:</u> If Player 'O' or Player 'X' can win **ANY** two rounds (excluding the consecutive first two rounds in Case 1), display the following output:

Player O can win two out of three rounds in the game.
Player O is the winner.

<u>Case 3:</u> Or else, please output the following if **NO** player can win the game:

No Player can win two out of three rounds in the game.
The game was a draw.

(d) Ask human users if they would like to run the game again. If yes, repeat (a) ~ (c). If not, terminate the program and then display "Thank You for Playing Our Game".

Note:

- The following is the sample output. All the **red bold prompts** are entered by the user.
- Except for the player selection, you can assume that all the users' inputs are valid.
- You don't need to consider if there are any typos, case sensitive, etc., in the inputs.

Here is the Sample Output:

```
Player Selection:
1. Random Player
2. MiniMax Player
3. Alpha Beta Player
4. Heuristic Alpha Beta Player
5. MCTS Player
6. Query Player

Please enter your first player: 0

Could not find your player, please try again: 1

Please enter your second player: 7

Could not find your player, please try again: 6

Round 1:
Available Action by the Player X:  [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]

The Action by the Player X Is (2, 3)
. . .
. . X
. . .

Player X's Utility:  0

current state:
. . .
. . X
. . .

available moves: [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (3, 1), (3, 2), (3, 3)]

Your move? (1, 1)
O . .
. . X
. . .

Player O's Utility:  0

O X .
. . X
. . .

Player X's Utility:  0

current state:
O X .
. . X
. . .

available moves: [(1, 3), (2, 1), (2, 2), (3, 1), (3, 2), (3, 3)]

Your move? (3, 3)
O X .
. . X
. . O
```

Player O's Utility: 0

O X .
X . X
. . O

Player X's Utility: 0

current state:
O X .
X . X
. . O

available moves: [(1, 3), (2, 2), (3, 1), (3, 2)]

Your move? **(2, 2)**
O X .
X O X
. . O

Player O's Utility: 1
Player O won the game in Round 1

Round 2:
Available Action by the Player X: [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]

The Action by the Player X Is (3, 1)
. . .
. . .
X . .

Player X's Utility: 0

current state:
. . .
. . .
X . .

available moves: [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 2), (3, 3)]

Your move? **(1, 3)**
. . O
. . .
X . .

Player O's Utility: 0

. . O
. . .
X . X

Player X's Utility: 0

current state:
. . O
. . .
X . X

available moves: [(1, 1), (1, 2), (2, 1), (2, 2), (2, 3), (3, 2)]

Your move? **(3, 2)**
. . O
. . .
X O X

Player O's Utility: 0

. . O
X . .
X O X

Player X's Utility: 0

current state:
. . O
X . .
X O X

available moves: [(1, 1), (1, 2), (2, 2), (2, 3)]

Your move? **(1, 1)**
O . O
X . .
X O X

Player O's Utility: 0

O . O
X . X
X O X

Player X's Utility: 0

current state:
O . O
X . X
X O X

available moves: [(1, 2), (2, 2)]

Your move? **(2, 2)**
O . O
X O X
X O X

Player O's Utility: 0

O X O
X O X
X O X

Player X's Utility: 0
Player X and Player O drew the game in Round 2.

Round 3
Available Action by the Player X: [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]

The Action by the Player X Is (1, 3)
. . X
. . .
. . .

Player X's Utility: 0

current state:
. . X
. . .
. . .

available moves: [(1, 1), (1, 2), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]

Your move? **(3, 3)**
. . X
. . .
. . O

Player O's Utility: 0

. . X
. . X
. . O

Player X's Utility: 0

current state:
. . X
. . X
. . O

available moves: [(1, 1), (1, 2), (2, 1), (2, 2), (3, 1), (3, 2)]

Your move? **(3, 1)**
. . X
. . X
O . O

Player O's Utility: 0

X . X
. . X
O . O

Player X's Utility: 0

current state:
X . X
. . X
O . O

available moves: [(1, 2), (2, 1), (2, 2), (3, 2)]

Your move? **(1, 2)**
X O X
. . X
O . O

Player O's Utility: 0

X O X
. . X
O X O

Player X's Utility: 0

current state:
X O X
. . X
O X O

available moves: [(2, 1), (2, 2)]

Your move? **(2, 2)**
X O X
. O X
O X O

Player O's Utility: 0

X O X
X O X
O X O

Player X's Utility: 0
Player X and Player O drew the game in Round 3.

No Player can win two out of three rounds in the game.
The game was a draw.

Would you like to play the game again? **No**
Thank You for Playing Our Game!

Grading Criteria: Your answers must be complete and clear.

Checkpoints	Points Possible
(1). Proper Naming Conventions and Program Documentation on Your Codes	10 Points
(2). Compliable Codes: ▪ TicTacToeClass.py ▪ TicTacToeGameApp.py ▪ Other python files/folders (if needed)	15 Points
(3). 6 Player Selection: ▪ Random Player ▪ MiniMax Player ▪ Alpha Beta Player ▪ Heuristic Alpha Beta Player ▪ MCTS Player ▪ Query Player	5 Points
(4). Minimax Search Implementation	10 Points

(5). Alpha-Beta Search Implementation	10 Points
(6). Alpha-Beta Cutoff Search Implementation	10 Points
(7). Monte Carlo Search Implementation	10 Points
(8). Prompt and ask for the player selection	5 Points
(9). In case of an error choosing the wrong player, ask the user to enter them again until the two players are valid	5 Points
(10). Create a TTT object from the TicTacToeClass.py based on the user's input above.	5 Points
(11). Show each move of player 'X' or 'O' in its own turn, by running their own searching strategy, respectively, in each individual game.	5 Points
(12). Ask the human user if he/she wants to run the game again. If yes, repeat the above steps. If not, terminate the program and then display "Thank You for Playing Our Game".	5 Points
(13). Your running output generated from your program should be the same as the sample output.	5 Points
Total	100 Points